

Sistema de Estacionamiento UCN CQBO

Informe Técnico: Análisis de Requisitos y Especificación de Diseño

Equipo de Desarrollo

Universidad Católica del Norte (UCN) - Sede Coquimbo (CQBO)

19 de mayo de 2026

Capítulo 1

Análisis de Requisitos

1.1. Introducción

El Sistema de Estacionamiento UCN CQBO requiere definir claramente cómo se gestiona el estado de los espacios de estacionamiento, quién puede realizar cambios y cómo se registra la información histórica.

1.2. Entes habilitados para cambiar el estado de un espacio

El cambio de estado de un espacio puede realizarse por **múltiples actores**:

- **Sistema automático (sensores IoT)**: Cambios automáticos cuando los sensores de presencia detectan ocupación o desocupación
- **Administradores**: Cambios manuales a través de API o dashboard para corregir inconsistencias

1.2.1. Rango de Autorización

Rol	Lectura	Cambio de Estado
Usuario normal	Sí	No
Staff	Sí	No
Admin	Sí	Sí
Sistema (sensores)	Sí	Sí*

Cuadro 1.1: Matriz de autorización por rol (*con validación especial)

1.2.2. Implicaciones

- Se requiere un campo `actualizado_por` en la entidad `Espacio` para auditoría
- Validación de JWT para cambios manuales
- Validación especial (header `X-Cambio-Por`) para cambios automáticos del sistema
- Los usuarios normales NO tienen acceso directo a cambiar estados

1.3. Registrar historial de entrada/salida y el estado actual

Ambos: estado actual + historial opcional en fase 2.

1.3.1. Implementación

La solución adopta un modelo de capas:

1. Capa 1 - Estado Actual

- Tabla **Espacio**: mantiene el estado vigente
- Campos: `id`, `sector_id`, `estado`, `actualizado_por`, `actualizado_en`
- Consultas rápidas para disponibilidad
- Rendimiento optimizado

2. Capa 2 - Historial Completo

- Tabla **HistorialEspacio**: registra cambios históricos
- Campos: `id`, `espacio_id`, `estado_anterior`, `estado_nuevo`, `actualizado_por`, `timestamp`, `observaciones`
- Auditoría completa
- Análisis de patrones de ocupación
- No afecta rendimiento de consultas de disponibilidad

1.3.2. Ventajas del Diseño

- **MVP rápido**: solo tabla **Espacio** en v1
- **Escalable**: historial separado sin overhead
- **Analítica**: datos históricos disponibles para estadísticas
- **Rendimiento**: consultas de disponibilidad no degradadas

1.4. Reserva de espacios por usuarios o solo administradores

Solo ocupación instantánea. No hay sistema de reservas.

1.4.1. Decisión

- No existe modelo de “pre-reservas” de espacios
- Un espacio está **disponible** u **ocupado** en tiempo real
- Los cambios se reflejan inmediatamente
- Modelo más simple y directo

1.4.2. Estados Permitidos

Estado	Significado
disponible	Espacio libre y listo para uso
ocupado	Espacio en uso actualmente

Cuadro 1.2: Estados permitidos de un espacio

1.4.3. Estados No Implementados

- reservado: No hay preemptive reservations
- mantenimiento: No considerado en fase MVP
- deshabilitado: No considerado en fase MVP

1.4.4. Implicaciones

- Eliminación del estado **reservado** de los enums
- Modelo de datos simplificado
- Sin tabla **Reserva**
- Sin validaciones complejas de conflictos de reservas

1.4.5. Tabla de Registro/Historial

No requerida inicialmente. Solo estado actual es suficiente para MVP.

1.4.6. Justificación

- El estado actual en tabla **Espacio** es suficiente para operación diaria
- Historial opcional en **HistorialEspacio** si se necesita auditoría
- Decisión orientada a MVP: minimalista y directo
- Escalable para futuras expansiones sin una reestructuración mayor

1.4.7. Enfoque

“Comenzar simple, expandir cuando sea necesario”

1.5. Matriz Resumen de Decisiones

Aspecto	Decisión	Razón
Control de estado	Multi-actor (sistema + admins)	Flexibilidad y automatización
Historial	Opcional (estado + historial)	MVP simple, escalable
Reservas	No (ocupación instantánea)	Menor complejidad
Tabla Registro	No (solo estado actual)	MVP minimalista
Estados	disponible, ocupado	Suficiente para requerimientos

Cuadro 1.3: Matriz de decisiones arquitectónicas

Capítulo 2

Especificación de Diseño

2.1. Entidades del Sistema

2.1.1. Entidad: Sector

Representa un área de estacionamiento en el campus.

Campo	Tipo	Descripción
id	str (pk)	Identificador único (ej: “A”, “B”)
nombre	str	Nombre del sector (ej: “Sector Guacolda”)
imagen	str (nullable)	URL o ruta a imagen de fondo del mapa
creado_en	datetime	Timestamp de creación
actualizado_en	datetime	Timestamp de última actualización

Cuadro 2.1: Estructura de entidad Sector

2.1.2. Entidad: Espacio

Representa un espacio de estacionamiento individual dentro de un sector.

Campo	Tipo	Descripción
id	str (pk)	ID único composición: sector_id + numero (ej: “A1”)
sector_id	str (fk)	Referencia al sector contenedor
estado	Enum	disponible — ocupado
actualizado_por	str	“sistema” — “admin_id” — email
actualizado_en	datetime	Timestamp del último cambio de estado

Cuadro 2.2: Estructura de entidad Espacio

2.1.3. Entidad: Usuario

Representa un usuario del sistema (administrador, staff o usuario normal).

Campo	Tipo	Descripción
id	int (pk)	Identificador único autoincrementado
nombre	str	Nombre completo del usuario
email	str (unique)	Email único para login
password_hash	str	Hash bcrypt de contraseña
role	Enum	admin — staff — user
activo	bool	Indica si el usuario está habilitado
creado_en	datetime	Timestamp de creación de cuenta

Cuadro 2.3: Estructura de entidad Usuario

2.1.4. Entidad: HistorialEspacio

Tabla de auditoría para registrar cambios históricos de estados.

Campo	Tipo	Descripción
id	int (pk)	Identificador único autoincrementado
espacio_id	str (fk)	Referencia al espacio
estado_anterior	Enum	Estado previo
estado_nuevo	Enum	Nuevo estado
actualizado_por	str	Quién/qué realizó el cambio
timestamp	datetime	Cuándo ocurrió el cambio
observaciones	str (nullable)	Notas sobre el cambio (ej: "Sensor desincronizado")

Cuadro 2.4: Estructura de entidad HistorialEspacio (opcional)

2.2. Arquitectura de Control de Acceso

2.2.1. Cambio de Estado por Sistema (Sensor Automático)

Listing 2.1: Flujo de cambio automático

```

1. Sensor IoT detecta ocupaci n/desocupaci n
2. Env a request a API: PUT /espacios/{espacio_id}/estado
3. Header especial: X-Cambio-Por: sistema
4. API valida que sea del sistema (IP, API key, etc.)
5. Actualiza: estado + actualizado_por="sistema" + timestamp
6. Opcional: guarda en HistorialEspacio
7. Retorna estado actualizado

```

2.2.2. Cambio de Estado Manual (Admin)

Listing 2.2: Flujo de cambio manual

```

1. Admin accede al dashboard del sistema
2. Selecciona espacio y nuevo estado
3. API recibe: PUT /espacios/{espacio_id}/estado
4. Valida JWT (usuario autenticado es admin)
5. Extrae user_id del token

```

6. Actualiza: estado + actualizado_por="admin_<id>" + timestamp
7. Opcional: guardar observaciones en historial
8. Retorna confirmaci n

2.2.3. Consulta de Disponibilidad (Público)

Listing 2.3: Flujo de consulta

1. Usuario accede a sectores.html (p blico)
2. GET /sectores (sin autenticaci n)
3. Retorna lista de todos los sectores
4. GET /sectores/{sector_id}
5. Retorna sector con estado actual de espacios
6. Frontend renderiza mapa interactivo:
 - Verde: disponible
 - Rojo: ocupado
7. Datos se actualizan en tiempo real (polling o WebSocket)

2.3. Schemas Pydantic

Los schemas Pydantic proporcionan validación automática y documentación de APIs.

2.3.1. Schemas de Espacio

Listing 2.4: Schema EspacioUpdateEstado

```
class EspacioUpdateEstado(BaseModel):
    """Schema para cambiar estado (solo admins/sistema)."""
    estado: Estado # 'disponible' | 'ocupado'
    observaciones: Optional[str] = None

class EspacioOut(BaseModel):
    id: str
    estado: Estado
    actualizado_por: str # 'sistema' | 'admin_<id>'
    actualizado_en: datetime

    class Config:
        orm_mode = True
```

2.3.2. Schemas de Sector

Listing 2.5: Schema SectorOut

```
class SectorOut(BaseModel):
    id: str
    nombre: str
    imagen: Optional[str]
    creado_en: datetime
    actualizado_en: datetime
```



```
espacios: List[EspacioOut]

class Config:
    orm_mode = True
```

2.3.3. Schemas de Usuario

Listing 2.6: Schema UserOut

```
class UserOut(BaseModel):
    id: int
    nombre: str
    email: EmailStr
    role: Role # 'admin' | 'staff' | 'user'
    activo: bool
    creado_en: datetime

class Config:
    orm_mode = True
```